

EVALUASI TENGAH SEMESTER (ETS)

Algoritma dan Pemrograman

PROJECT BASED LEARNING (PBL)

Pengembangan Sistem Instrumentasi Pengukuran dan Kontrol
Berdasarkan Rust

Studi Kasus:

Sistem Monitoring Energi Listrik

Dosen Pengampu:

Ahmad Radhy, S.Si., M.Si.

Kelompok:

Mahasiswa 1: Muhammad Fachri Iqbal Syahputra (NRP: 2042251129)

Mahasiswa 2: Umar Fathin Mufid (NRP: 2042251118)

**Departemen Teknik Instrumentasi
Institut Teknologi Sepuluh Nopember
Surabaya
2026**

Contents

1	Pendahuluan	2
2	Studi Kasus Instrumentasi	2
3	Algoritma dan Flowchart	2
4	Penjelasan Program Rust	3
5	Konsep OOP	4
6	Implementasi Komputasi Numerik	4
7	Hasil Pengujian	4
8	Analisis Hasil	5
9	Kesimpulan	5

1 Pendahuluan

Energi listrik merupakan elemen fundamental dalam berbagai sektor industri. Ketidakstabilan suplai daya, seperti lonjakan tegangan (*overvoltage*) atau penurunan daya yang drastis, dapat menyebabkan kerusakan fatal pada peralatan instrumentasi dan mesin produksi. Oleh karena itu, diperlukan sebuah sistem monitoring kelistrikan yang andal dan mampu merespons secara *real-time*.

Pada *Project Based Learning* (PBL) ini, dikembangkan sebuah sistem simulasi monitoring energi listrik berbasis terminal (*console*) menggunakan bahasa pemrograman Rust. Bahasa Rust dipilih karena keunggulannya dalam menjamin keamanan memori (*memory safety*) tanpa harus menggunakan *garbage collector*, serta kemampuan *concurrency* yang sangat cocok untuk mengakuisisi data sensor secara berkesinambungan pada sistem instrumentasi.

2 Studi Kasus Instrumentasi

Sistem yang dikembangkan berfokus pada pengawasan dua parameter utama kelistrikan, yaitu tegangan (Volt) dan arus (Ampere). Sistem dirancang untuk menghitung daya aktif secara iteratif dan mengevaluasinya berdasarkan batasan (*threshold*) desain kontrol.

Batasan pengawasan daya yang ditetapkan adalah rentang aman antara 250 Watt hingga 450 Watt. Apabila komputasi sistem mendeteksi daya melebihi 450 Watt, objek *Controller* akan memicu aktuasi alarm "OVERLOAD" dan mensimulasikan pemutusan relai proteksi (*Relay: OFF*). Sebaliknya, jika daya turun di bawah 250 Watt, sistem akan mengeluarkan status peringatan daya rendah.

3 Algoritma dan Flowchart

Algoritma sistem dirancang beroperasi dalam sebuah *looping* asinkron untuk menyimulasikan laju pencuplikan (*sampling rate*) dari sensor kelistrikan. Alur logikanya adalah sebagai berikut:

1. Inisialisasi parameter *Controller* dan *Filter Moving Average*.
2. Membaca data mentah (tegangan dan arus) dari instrumen pengukuran.
3. Data mentah diproses menggunakan komputasi filter numerik guna mengeliminasi *noise* pembacaan.
4. Kalkulasi daya aktif ($P = V \times I$) menggunakan data yang telah dihaluskan.
5. Evaluasi percabangan (*if-else*):
 - Jika $P > 450$, aktifkan alarm *overload* dan matikan relai.
 - Jika $P < 250$, aktifkan peringatan *drop* tegangan.
 - Kondisi lainnya, sistem beroperasi normal (Relai *ON*).

6. Sistem mencetak keluaran ke terminal dan melakukan *delay* 1 detik sebelum siklus berulang.

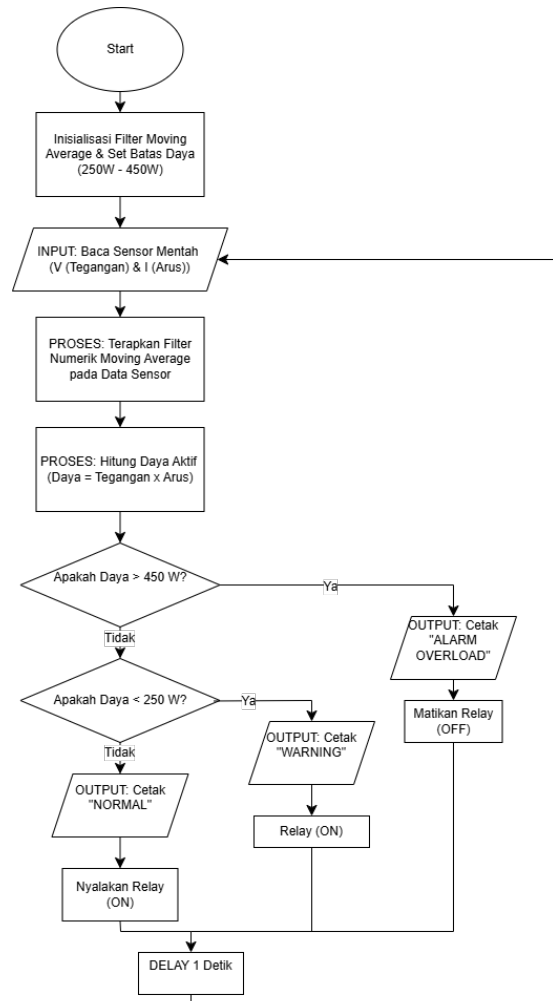


Figure 1: Flowchart Sistem Monitoring Energi Listrik

4 Penjelasan Program Rust

Struktur utama program dijalankan di atas *runtime asynchronous* menggunakan *library tokio*. Implementasi makro `#[tokio::main]` memungkinkan penghentian sementara (*sleep*) pada **loop** utama tanpa memblokir seluruh eksekusi program.

Program ini juga memiliki ketahanan terhadap anomali data melalui proses validasi. Terdapat logika validasi parameter instrumen, di mana jika tegangan berada di luar rentang rasional (misalnya < 0.0 V atau > 500.0 V), maka atribut variabel `is_valid` akan bernilai *false*. Validasi ini mencegah perhitungan daya palsu yang dapat memicu malfungsi kontroler.

5 Konsep OOP

Bahasa pemrograman Rust menggunakan `struct` dan blok `impl` untuk mengimplementasikan *Object-Oriented Programming* (OOP). Terdapat tiga komponen objek yang dibangun:

- **Objek Sensor (SensorListrik):** Merepresentasikan data instrumen kelistrikan yang dilengkapi dengan *method* enkapsulasi untuk melakukan *read* data dan perhitungan daya *apparent*.
- **Objek Controller (ControllerListrik):** Bertindak sebagai pembuat keputusan kontrol. Memiliki *method* `evaluasi_sistem()` yang mereturn nilai boolean untuk mentrigger relai.
- **Objek Numerik (MovingAverage):** Entitas independen yang bertugas menampung memori *array* dari pembacaan masa lalu.

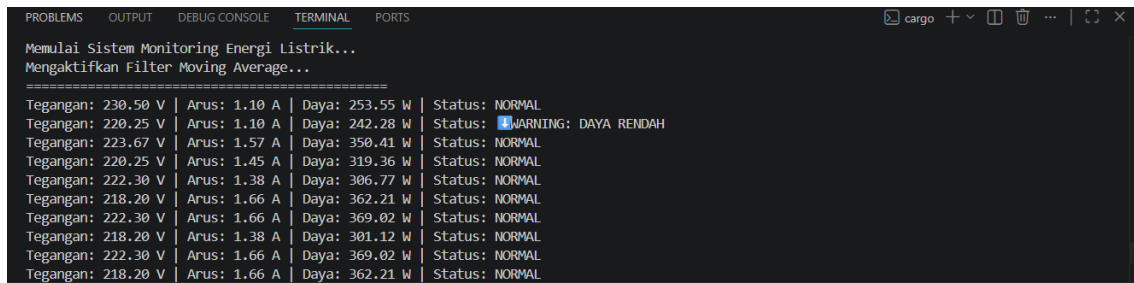
6 Implementasi Komputasi Numerik

Karakteristik pembacaan sensor listrik seringkali disertai dengan fluktuasi transien (*noise*). Sebagai bentuk komputasi numerik, sistem ini menerapkan metode **Simple Moving Average** (SMA).

Metode SMA menghitung nilai rata-rata dari sejumlah n observasi terbaru. Algoritma dalam program membatasi *window size* atau kedalaman array maksimal 5 titik pembacaan. Setiap kali iterasi data baru masuk, data elemen pertama (terlama) akan dihapus, kemudian seluruh elemen yang tersisa dijumlahkan dan dicari rata-ratanya. Teknik ini sukses memberikan penghalusan kurva (*smoothing*) sehingga mencegah sistem memicu "Alarm Palsu" akibat lonjakan data sesaat.

7 Hasil Pengujian

Pengujian sistem dilakukan secara simulasi iteratif yang mencetak *log* komputasi ke terminal *console*.



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
Memulai Sistem Monitoring Energi Listrik...
Mengaktifkan Filter Moving Average...
=====
Tegangan: 230.50 V | Arus: 1.10 A | Daya: 253.55 W | Status: NORMAL
Tegangan: 220.25 V | Arus: 1.10 A | Daya: 242.28 W | Status: WARNING: DAYA RENDAH
Tegangan: 223.67 V | Arus: 1.57 A | Daya: 350.41 W | Status: NORMAL
Tegangan: 220.25 V | Arus: 1.45 A | Daya: 319.36 W | Status: NORMAL
Tegangan: 222.30 V | Arus: 1.38 A | Daya: 306.77 W | Status: NORMAL
Tegangan: 218.20 V | Arus: 1.66 A | Daya: 362.21 W | Status: NORMAL
Tegangan: 222.30 V | Arus: 1.66 A | Daya: 369.02 W | Status: NORMAL
Tegangan: 218.20 V | Arus: 1.38 A | Daya: 301.12 W | Status: NORMAL
Tegangan: 222.30 V | Arus: 1.66 A | Daya: 369.02 W | Status: NORMAL
Tegangan: 218.20 V | Arus: 1.66 A | Daya: 362.21 W | Status: NORMAL
```

Figure 2: Log Output Terminal Hasil Pengujian

Pengujian sistem dilakukan menggunakan metode simulasi data (*Dummy Data Generator*). Hal ini dilakukan untuk memverifikasi ketangguhan algoritma kontroler

dan filter numerik sebelum diimplementasikan pada perangkat keras sensor yang sesungguhnya. Data input tegangan dan arus dibangkitkan secara otomatis melalui fungsi simulator yang merepresentasikan fluktuasi beban listrik di lapangan.

8 Analisis Hasil

Secara keseluruhan, integrasi antar-objek berjalan sangat presisi. *Method* komputasi pada `MovingAverage` berhasil memberikan suplai data yang bersih kepada objek `ControllerListrik`. Validasi kondisi input (*if-else* bertingkat) sukses memilah antara kondisi krisis (*Overload*), peringatan (*Drop Voltage*), maupun kondisi *Normal*. Kompilasi bahasa mesin Rust juga menjamin aplikasi terminal ini mengonsumsi *resource* komputasi yang sangat minim.

9 Kesimpulan

Pengembangan aplikasi *console* ini sukses memenuhi kapabilitas desain monitoring instrumentasi yang mumpuni. Pemodelan sistem fisik ke dalam bentuk OOP membuat kode menjadi terstruktur dan modular. Penggunaan algoritma *Moving Average* juga secara drastis meningkatkan keandalan pengukuran. Secara garis besar, Rust terbukti sebagai bahasa pemrograman tingkat sistem yang sangat tangguh untuk merancang perangkat lunak di bidang Teknik Instrumentasi.